

Práctica 4

Procesamiento e Interpolación de Imágenes Táctiles para un Gripper Robótico utilizando C++ y Python

Grado en Tecnologías Digitales para la Empresa
Asignatura: Sistemas Operativos

Dr. Jaime Andrés Rincón Arango

Curso 2025-2026

1. Contextualización

En los sistemas robóticos modernos, los sensores táctiles permiten que los robots perciban propiedades físicas del entorno, como presión, textura, forma y distribución de contacto. Este tipo de percepción es especialmente importante en aplicaciones de manipulación robótica, donde los grippers deben interactuar con objetos frágiles, deformables o de geometría compleja.

Los sensores táctiles matriciales funcionan de manera similar a una imagen en escala de grises, donde cada celda representa el nivel de presión detectado en una posición específica. Sin embargo, muchos sensores de bajo coste poseen resoluciones reducidas, por ejemplo matrices de 16×16 , lo que limita el nivel de detalle disponible.

En esta práctica se trabajará con un sensor táctil integrado en un gripper robótico. Los datos capturados han sido almacenados en formato JSON y contienen 50 capturas táctiles diferentes. A partir de estas matrices, los estudiantes deberán aplicar técnicas de interpolación para aumentar la resolución hasta 128×128 , permitiendo generar imágenes táctiles más detalladas.

Posteriormente, el sistema desarrollado en C++ deberá enviar las matrices interpoladas a un servidor en Python mediante comunicación HTTP, donde finalmente se generarán imágenes utilizando Matplotlib.

Esta práctica integra conceptos fundamentales de:

- Procesamiento de matrices.
- Interpolación numérica.
- Comunicación cliente-servidor.
- Procesamiento de señales táctiles.
- Visualización científica.

2. Objetivos

Objetivo general

Desarrollar un sistema distribuido en C++ y Python capaz de procesar información táctil proveniente de un sensor robótico, aumentando su resolución mediante interpolación y generando representaciones visuales de la presión detectada.

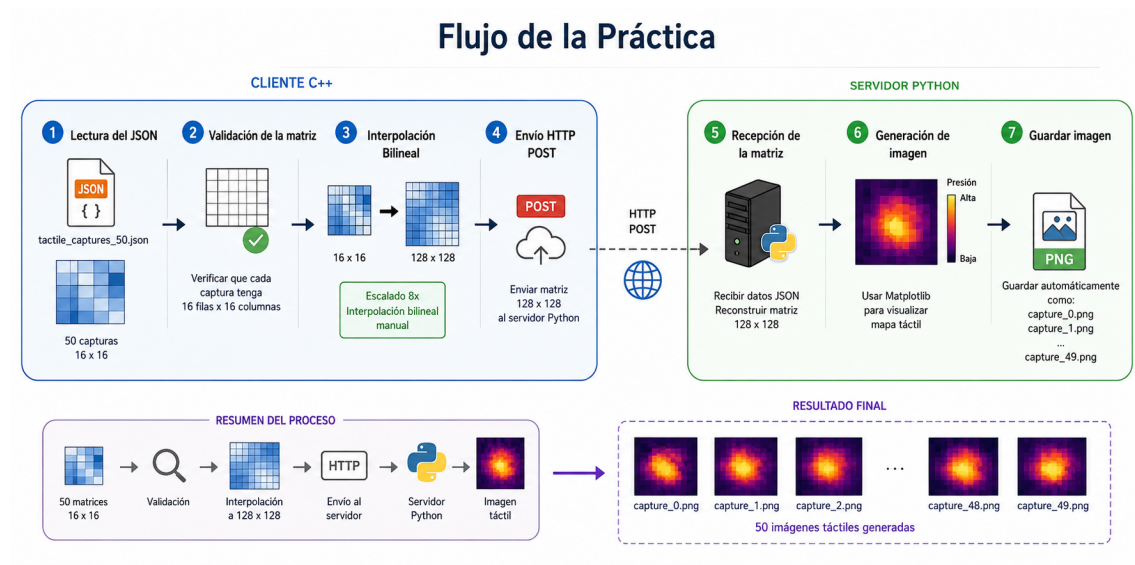


Figura 1: Flujo general de la práctica para el procesamiento de imágenes táctiles en un gripper robótico. El sistema inicia con la lectura de matrices táctiles 16×16 almacenadas en formato JSON, seguido de la validación de dimensiones, interpolación bilineal hasta 128×128 , envío de datos mediante HTTP desde un cliente en C++ hacia un servidor Python, y generación automática de mapas táctiles utilizando Matplotlib.

Objetivos específicos

Al finalizar la práctica, el estudiante deberá ser capaz de:

- Leer y procesar archivos JSON en C++.
- Manipular matrices bidimensionales.
- Implementar interpolación bilineal para escalado de imágenes.
- Desarrollar un cliente HTTP en C++.
- Implementar un servidor HTTP en Python.
- Generar imágenes científicas utilizando Matplotlib.

3. Descripción de la práctica

Se dispone de un conjunto de datos táctiles almacenados en un archivo JSON. Cada captura representa una matriz de presión de tamaño:

$$16 \times 16$$

Cada captura contiene exactamente:

$$16 \text{ filas} \times 16 \text{ columnas}$$

para un total de:

$$256 \text{ valores por captura}$$

Cada elemento de la matriz contiene un valor numérico correspondiente a la intensidad de presión detectada por el sensor táctil del gripper.

Los estudiantes deberán:

1. Leer las 50 capturas desde el archivo JSON.
2. Validar que todas las matrices tengan dimensiones 16×16 .
3. Convertir cada matriz de 16×16 a una matriz de 128×128 .
4. Aplicar interpolación bilineal.
5. Enviar cada matriz interpolada a un servidor Python.
6. Generar una imagen táctil usando Matplotlib.
7. Guardar las imágenes generadas.

4. Estructura del archivo JSON

El archivo JSON tendrá la siguiente estructura:

```
{
  "captures": [
    {
      "id": 0,
      "matrix": [
        [0, 12, 14, 18, 20, 25, 30, 28, 18, 10, 5, 0, 0, 0, 0, 0],
        [0, 10, 15, 22, 30, 35, 40, 36, 25, 15, 8, 2, 0, 0, 0, 0],
        [0, 8, 18, 28, 40, 50, 60, 55, 40, 25, 12, 5, 0, 0, 0, 0],
        [0, 5, 15, 30, 50, 70, 85, 80, 60, 35, 20, 8, 2, 0, 0, 0],
        [0, 2, 10, 25, 45, 75, 100, 95, 70, 45, 25, 10, 4, 0, 0, 0],
        [0, 0, 8, 20, 40, 70, 95, 110, 90, 60, 35, 15, 6, 0, 0, 0],
        [0, 0, 5, 18, 35, 60, 90, 120, 100, 70, 40, 20, 8, 2, 0, 0],
        [0, 0, 4, 15, 30, 55, 85, 115, 105, 75, 45, 22, 10, 3, 0, 0],
        [0, 0, 3, 12, 25, 45, 70, 95, 90, 65, 40, 20, 8, 2, 0, 0],
        [0, 0, 2, 8, 18, 35, 55, 75, 70, 50, 30, 15, 6, 1, 0, 0],
        [0, 0, 0, 5, 12, 25, 40, 55, 50, 35, 20, 10, 4, 0, 0, 0],
        [0, 0, 0, 2, 8, 15, 25, 35, 30, 22, 12, 6, 2, 0, 0, 0],
        [0, 0, 0, 0, 4, 8, 12, 18, 16, 12, 6, 3, 0, 0, 0, 0],
        [0, 0, 0, 0, 2, 4, 6, 8, 7, 5, 3, 1, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 2, 3, 4, 3, 2, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 1, 2, 1, 0, 0, 0, 0, 0, 0, 0]
      ]
    }
  ]
}
```

Cada captura contiene:

- **id**: identificador de la captura.
- **matrix**: matriz táctil 16×16 .

El programa deberá validar que todas las capturas tengan exactamente 16 filas y 16 columnas.

5. Requisitos técnicos

Parte C++

El sistema cliente deberá:

- Leer el JSON.

- Procesar matrices.
- Aplicar interpolación bilineal.
- Enviar datos mediante HTTP POST.

Se recomienda utilizar:

- `nlohmann/json`
- `libcurl`

Parte Python

El servidor deberá:

- Recibir matrices vía HTTP.
- Reconstruir la matriz.
- Generar imágenes.

Librerías recomendadas:

```
Flask
NumPy
Matplotlib
```

6. Interpolación requerida

La interpolación deberá aumentar la resolución de:

$$16 \times 16 \rightarrow 128 \times 128$$

Factor de escala:

$$8x$$

Se deberá implementar interpolación bilineal de forma manual.

No está permitido utilizar funciones automáticas de OpenCV o SciPy para realizar directamente el escalado.

7. Comunicación cliente-servidor

El cliente en C++ deberá enviar las matrices utilizando HTTP POST.

Formato esperado:

```
{
  "capture_id": 0,
  "width": 128,
  "height": 128,
  "data": [[...]]
}
```

8. Visualización en Python

El servidor Python deberá generar una imagen táctil utilizando:

```
plt.imshow(matrix, cmap="inferno")
plt.colorbar(label="Presion")
plt.title("Mapa_tactil")
```

Las imágenes deberán guardarse automáticamente:

```
capture_0.png
capture_1.png
...
```

9. Entregables

1. Código fuente en C++

El desarrollo deberá realizarse obligatoriamente utilizando programación modular basada en funciones. El proyecto deberá estar correctamente organizado y separado en distintos archivos. Debe incluir:

- Lectura del archivo JSON.
- Validación de matrices 16×16 .
- Implementación manual de la interpolación bilineal.
- Cliente HTTP para el envío de datos.
- Uso obligatorio de funciones.
- Archivo de cabecera (*.h).
- Archivo de implementación (*.cpp).
- Archivo principal `main.cpp`.

Cada funcionalidad principal deberá implementarse mediante funciones independientes, por ejemplo:

- Función para leer el JSON.
- Función para validar matrices.
- Función para interpolar.
- Función para enviar datos HTTP.
- Función para convertir matrices a JSON.

No se permitirá desarrollar toda la práctica dentro de un único archivo `main.cpp`.

2. Código fuente en Python

Debe incluir:

- Servidor Flask.
- Generación de imágenes.

3. Informe técnico

Máximo 5 páginas.

Debe contener:

- Explicación del sistema.
- Explicación de la interpolación.
- Arquitectura cliente-servidor.
- Capturas de resultados.
- Problemas encontrados.
- Conclusiones.

4. Evidencias

- Capturas de pantalla.
- Imágenes generadas.
- Ejecución del sistema.

10. Resultados esperados

Al finalizar la práctica, se espera que el estudiante haya desarrollado un sistema completo capaz de:

- Procesar señales táctiles robóticas.
- Escalar matrices mediante interpolación.
- Implementar comunicación entre C++ y Python.

11. Rúbrica de evaluación

Criterio	Nota	Descripción
Lectura correcta del JSON	1.0	El sistema carga correctamente las 50 capturas y valida la estructura de los datos
Validación de matrices	1.0	El programa verifica correctamente que todas las matrices sean de 16×16
Implementación de interpolación	2.5	La interpolación bilineal está correctamente implementada y genera matrices válidas de 128×128
Comunicación HTTP	1.5	El cliente C++ envía correctamente los datos al servidor Python
Generación de imágenes	1.5	El servidor genera y almacena correctamente las imágenes táctiles
Modularidad y uso de funciones	1.5	El código utiliza correctamente funciones, separación en archivos <code>.h</code> , <code>.cpp</code> y estructura modular
Informe técnico	1.0	Claridad técnica, explicación adecuada y análisis de resultados

Total	10.0	
-------	------	--

12. Recomendaciones

- Implementar primero la lectura del JSON.
- Validar la interpolación antes de integrar el servidor.
- Probar el envío HTTP con matrices pequeñas.
- Verificar visualmente las imágenes generadas.
- Utilizar mapas de color adecuados para representar presión.

Fecha de entrega

La práctica deberá entregarse antes del:

2 Junio 2026

13. Material de apoyo recomendado

C++

- JSON for Modern C++
- libcurl documentation

Python

- Flask
- Matplotlib
- NumPy

